Project Phase 1: Battery Charger with MPPT Algorithm

ECE 128B Renewable Energy & Power Grid

Professor: Dr. Reza Esmaili

February 22, 2026

Barry Leung

A18484227

Charlie Frank

A16928173

Nolan Sim

A17434835

Table of Contents

Table of Contents **2**
Introduction **3**
Problem Definition **4**
Solution **5**
Figure 1: IV and PV characteristic schematic 5
Figure 2: Ipv vs Vpv 6
Figure 3: P vs Vpv: Maximum Power of ~2.25KW at ~450V 6
Figure 4: Schematic of current follower with PI Controller 7
Table 1: Current tracking parameters 7
Figure 5: Iout, Iref vs time for current tracking schematic 8
Figure 6: Duty cycle variation for switches U, V and W vs time in buck boost converter while current tracking 8
Figure 7: Schematic of voltage and current follower 9
Table 2: Voltage tracking parameters 9
Figure 8: Vpv , Vref vs time for voltage tracking schematic 9
Figure 9: Duty cycle variation for switches U, V and W vs tune in buck boost converter while voltage and current tracking 10
Figure 10: MPPT Implemented Schematic 10
Figure 11: Perturb and Observe code implemented into C-Block 11
Figure 12: Ppv, Vpv, and Ipv plot vs time for MPPT schematic 11
Conclusion **12**
Appendix A: C Block Code for MPPT control **13**

Introduction

The first phase of the project will focus on the design and simulation of a solar-powered battery charging system utilizing a three phase interleaved Buck-Boost converter. PSIM software simulations will be the primary tool used to accomplish these goals and report the system results. The primary objective of this system will be to determine efficient and stable operating points for transferring power from a photovoltaic (PV) array to a battery storage system under changing environmental conditions.

Due to variations in solar irradiance, the current–voltage (I–V) and power–voltage (P–V) characteristics will vary continuously. Thus, this control circuit setup will be required to achieve maximum power output. Without Maximum Power Point Tracking (MPPT), the efficiency of the PV system could be significantly reduced under certain solar conditions. An active control system will be implemented to regulate the operating parameters of the PV array and maintain maximum power point operation.

To ensure maximum efficiency is achieved, the Perturbation and Observation (P&O) method will be utilized as the MPPT algorithm. The MPPT algorithm will adjust the reference voltage to track the peak power operating point of the solar panel. These reference values will be sent through a proportional integral (PI) controller loop to generate duty cycle commands for the pulse width modulation (PWM) signals that control the converter switches. Through this control setup, the system will ensure optimal energy output while maintaining stable battery charging. A three phase Interleaved Buck-Boost converter will be used, as it allows both step up and step down voltage variation while also improving thermal management and reducing current ripple within the system.

Validation of this setup will be conducted by analyzing current and voltage tracking accuracy, dynamic system response, and steady state stability. Learning how to optimize such converters will be essential to implementing renewable energy storage systems in real world applications, especially in situations where solar power systems may have to deal with unpredictable weather conditions.

Problem Definition

Phase 1 of this project will solve the problem of extracting power from a solar array and transferring it to charge a 100V battery. The approach will involve the usage of a three phase interleaved Buck-Boost converter following a MPPT algorithm. In order to deal with variations in solar conditions, the system will also continuously implement the P&O method in order to adjust the operating point of the circuit.

The theoretical basis relies on managing specific electrical parameters. The solar panel will have a maximum open-circuit voltage of 600V and a short-circuit current of 6A. In order to be as efficient as possible, the P&O algorithm will focus on the maximum power point, which is at 450V and 5A, resulting in a max power output of 2250W. The converter will ensure the components operate together to reduce overall current ripple and thermal stress. As for physical restrictions, the design will have a fixed switching frequency of 10kHz as well as 50mΩ of internal resistance. In order to maintain stable operation, the inductors for each phase will be sized at 400μH, and the photovoltaic capacitor will be sized at 30μF.

The implementation of the control system will follow a structured sequence of steps. The methodology will begin with simulating the PV model characteristics and validation testing for the current and voltage control loops of the converter to ensure that the set up will perform as desired. In the final MPPT circuit, the algorithm block will generate a reference voltage from the solar array voltage and current measurements. Next, a voltage controller will compare the reference voltage to the actual panel voltage in order to generate a total reference current. This control structure will operate an outer voltage control loop and three inner current control loops. The reference current will be divided equally into three phase currents. Limiters and proportional–integral (PI) controllers will process the individual phase currents and provide control signals to the pulse width modulation (PWM) generators. The resulting PWM signals will control the switching states of the upper and lower switches in each interleaved phase leg. Together, these steps will accomplish the goal of designing an MPPT controller which can track changing environmental conditions to optimize system performance for this PV setup.

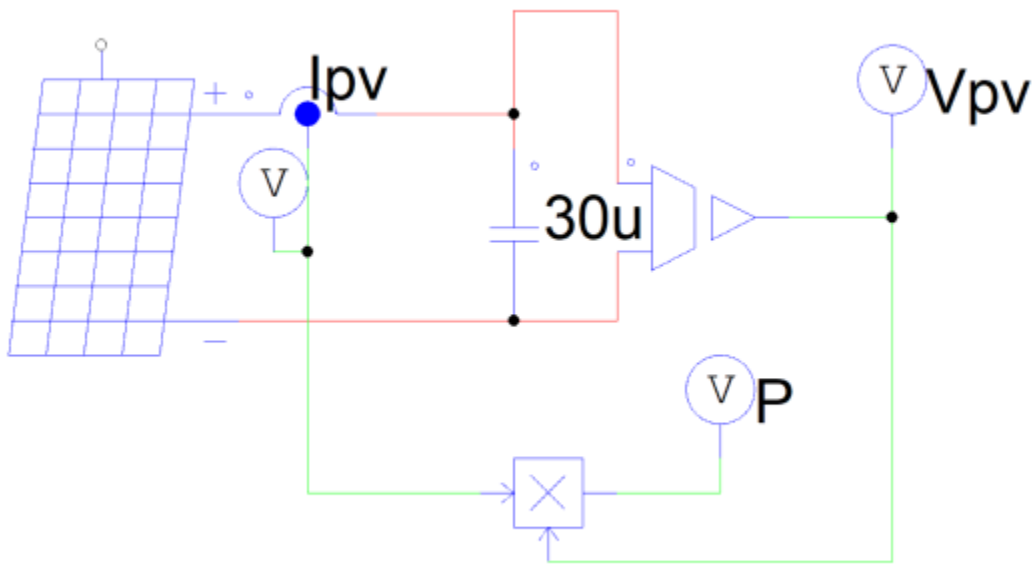
Solution

The battery charging system is constructed using the PSIM simulation software. A 3 phase interleaved Buck-Boost converter was created to convert 450VDC to 100VDC load to the battery. The specifications for the setup we simulated are as follows:

$$Vbat = 100VDC, \quad Cpv = 30uF, \quad R = 50m\Omega, \quad Fsw = 10kW$$
$$Voc = 600VDC, \quad Vmp = 450V, \quad Isc = 6A, \quad Imp = 5A$$

Using the MPP specifications given, we simulated the solar array with an output to a capacitor of the same specifications with no voltage transformations.

Figure 1: IV and PV characteristic schematic.



Using this schematic, we obtained the IV and PV curves for this particular solar array which we used as a reference for our MPPT algorithm to validate that it was tracking properly.

Figure 2: Ipv vs Vpv.

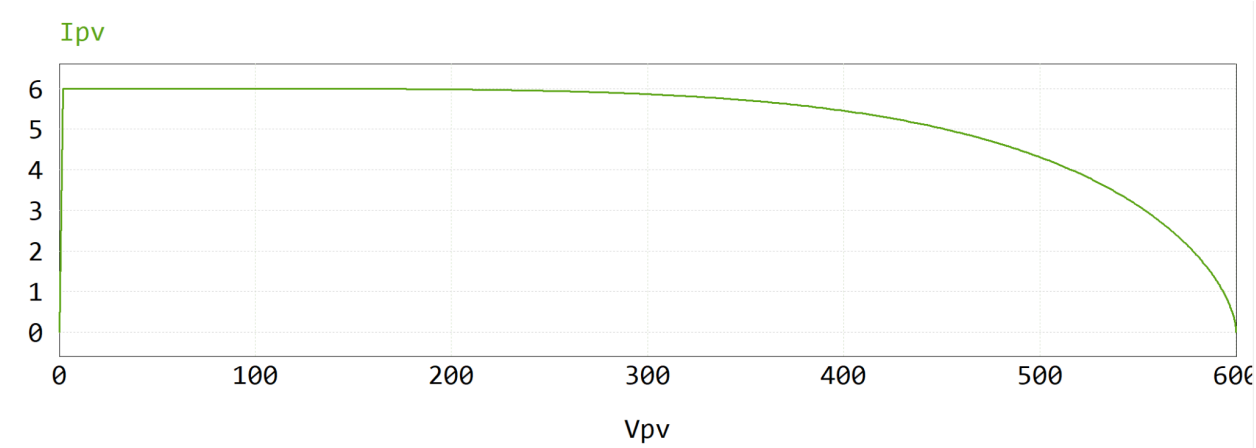
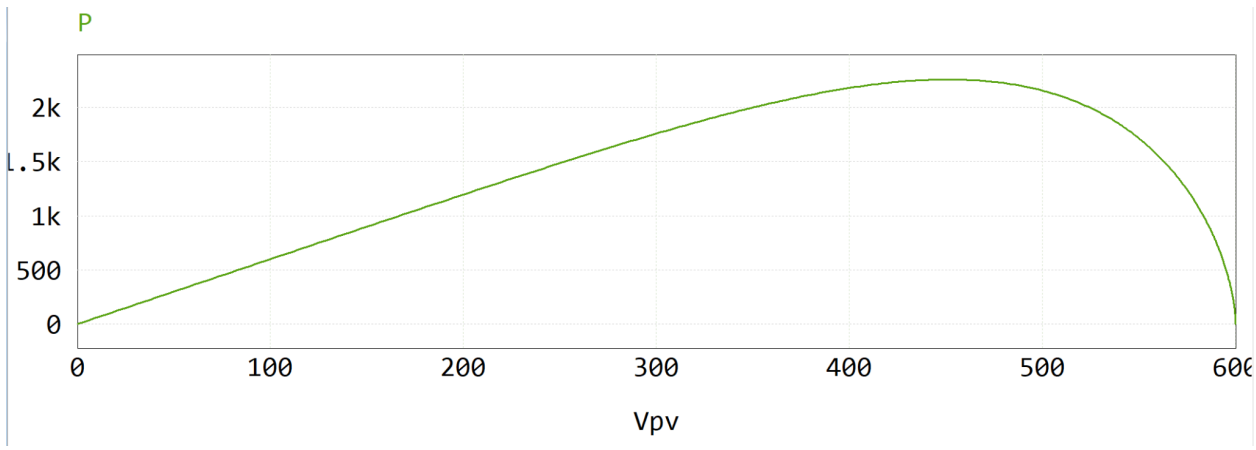


Figure 3: P vs Vpv, Maximum Power of ~2.25KW at ~450V.



Next, we implemented a proportional integral controller (PI) to our 3 phase interleaved Buck-Boost converter that tracked the current after a sudden voltage step. This voltage step was simulated with a Vstep input and is representative of the variance in voltage in PV systems. The PI controller adjusts the duty cycle of the switches that makes the Iout follow Iref. To correct overshoot and variance in duty cycle we adjust the gain and time constant in the PI controller. We settled on a gain of 0.01 and a time constant (tau) of 0.001, shown in Table 1, as having sufficient tracking of the Iref value while keeping variance of the duty cycle and output current low. We have also quantified the overshoot and variance in duty cycle in Table 1, along with other candidate gain and tau values. Other values of gain and tau were not used as lower percentages of current overshoot had a highly varying duty cycle and a tau too large made the circuit response to current variation slow.

Figure 4: Schematic of current follower with PI Controller.

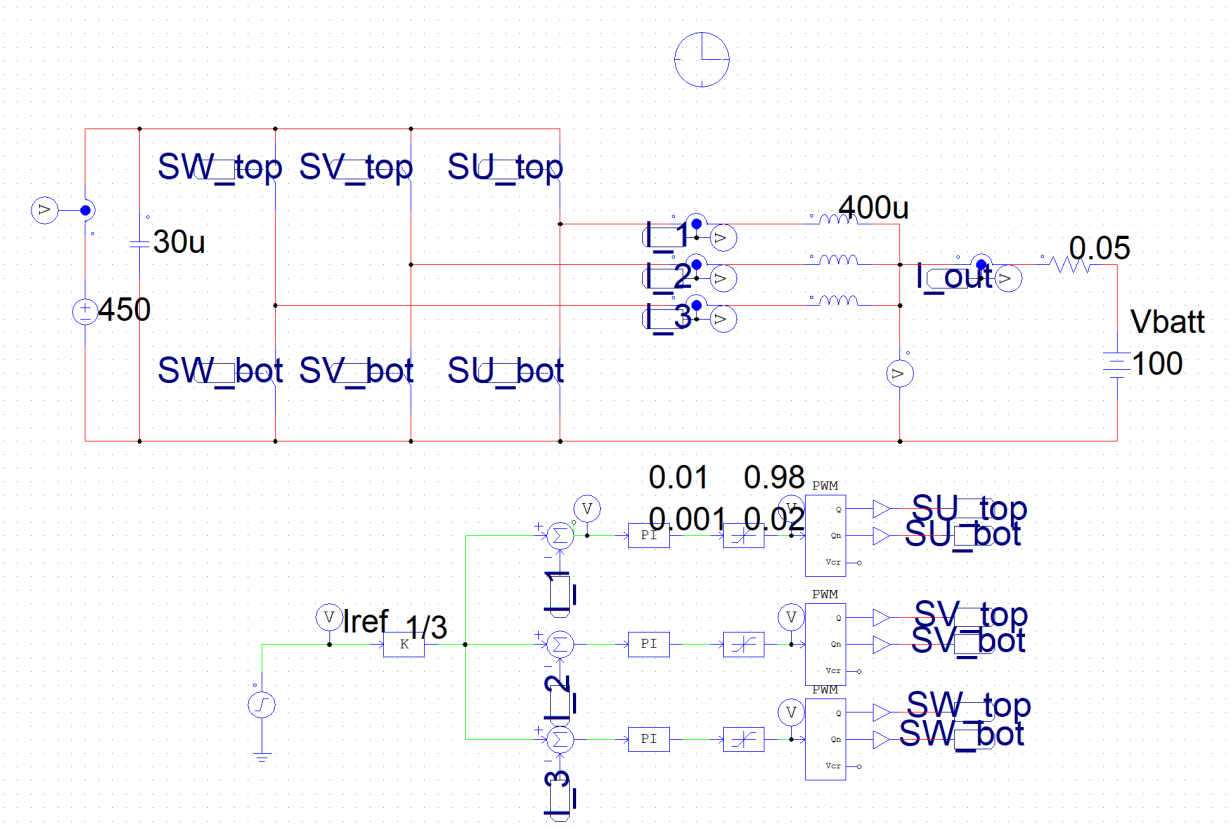


Table 1: Current tracking parameters.

Gain	Time Constant	Delta at steady state	Current Overshoot	Duty Cycle variance	Rise Time (s)
5	.001	+/- 2A	25%	.02-.98	~0
1	.001	+/- 2A	25%	.02-.98	~0
.01	.001	+/- 5A	30%	.22-.44	.005
.01	.01	+/- 6A	35%	.22-.44	.025

Figure 5: Iout, Iref vs time for current tracking schematic.

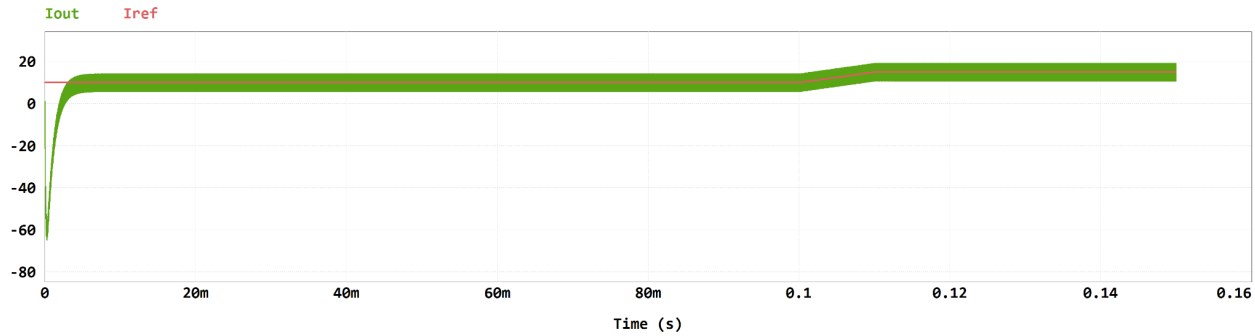
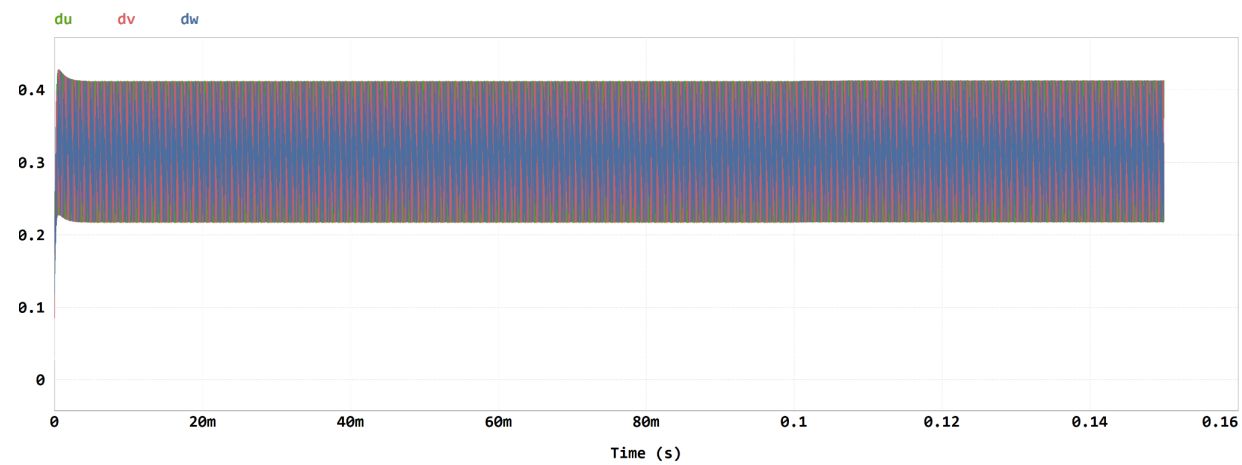


Figure 6: Duty cycle variation for switches U, V, and W vs time in buck boost converter while current tracking.



Next, we created a schematic that was able to track and respond to a change in voltage value. This is an important preliminary step before implementing our MPPT algorithm to ensure that our Vpv will track accordingly to a new reference voltage. To do this we have implemented another PI controller that is tracking the error between a voltage step from 450DC to 500VDC. The switches duty cycles are changed as a result of this PI controller tracking the Vpv to the Vref value. Appropriate gains and time constants are needed to ensure limited overshoot and settling time, while also allowing current controls enough time to respond to the voltage controller. We have quantified the overshoot and variance in duty cycle in the table below.

Figure 7: Schematic of voltage and current follower.

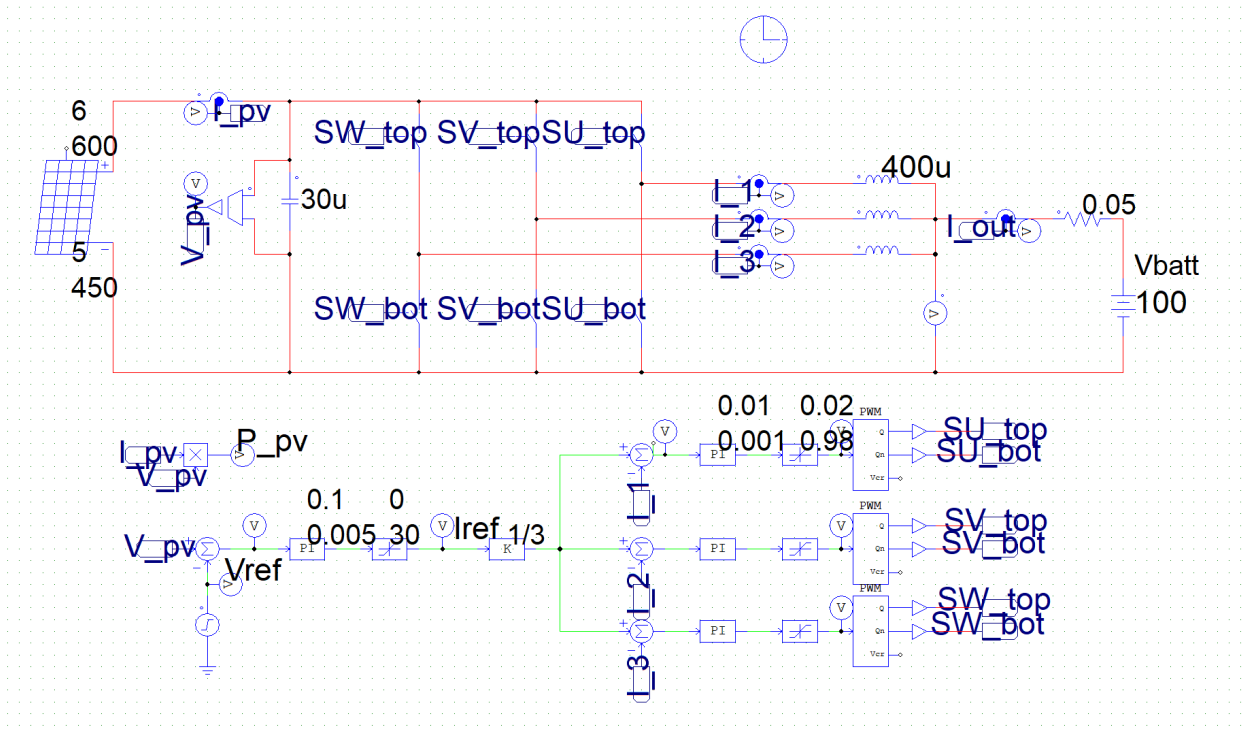


Table 2: Voltage tracking parameters.

Gain	Time Constant	Overshoot	Duty Cycle variance	Settling Time (s)	Rise time	Note
.1	.005	35.6%	.22-.44	.0225	.00625	
.1	0.01	35.6%	.22-.40	0.03	.00325	Slow to reach first Vref
10	.005	28.9%	.20-.50	0.007	~0	Vbatt dip after V change

Figure 8: Vpv , Vref vs time for voltage tracking schematic.

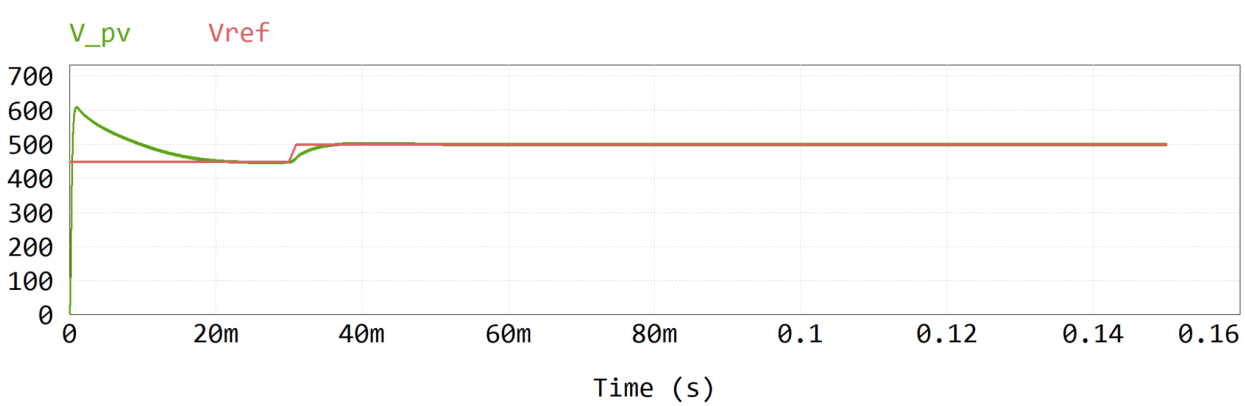
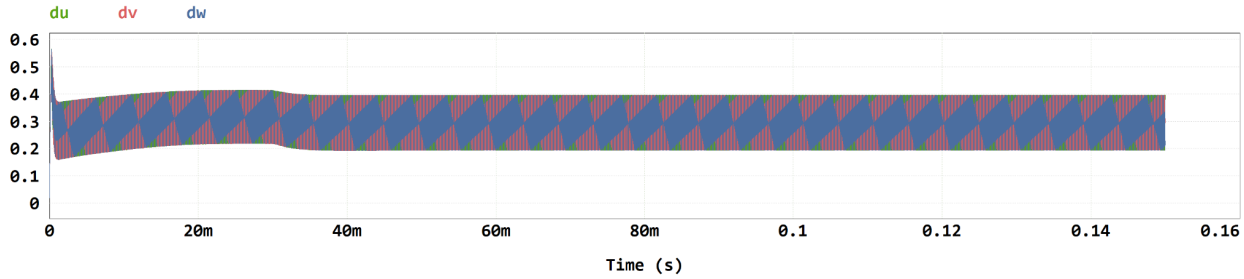


Figure 9: Duty cycle variation for switches U, V and W vs tune in buck boost converter while voltage and current tracking.



With a working current and voltage tracking schematic, we implemented our MPPT algorithm using the Perturb and Observe method (P&O). The power output was continuously calculated from the values of Ipv and Vpv (Equation 1). The basis of the P&O method is as follows, if a small disturbance in the voltage increases power output the voltage will continue to change in the same direction, if it decreases power it will reverse direction. The change in voltage is dictated by the varying duty cycles from the previous current and voltage regulators. The logic for the P&O method is implemented in a C block in the MPPT schematic.

$$Ppv = Ipv * Vpv \tag{1}$$

Figure 10: MPPT Implemented Schematic.

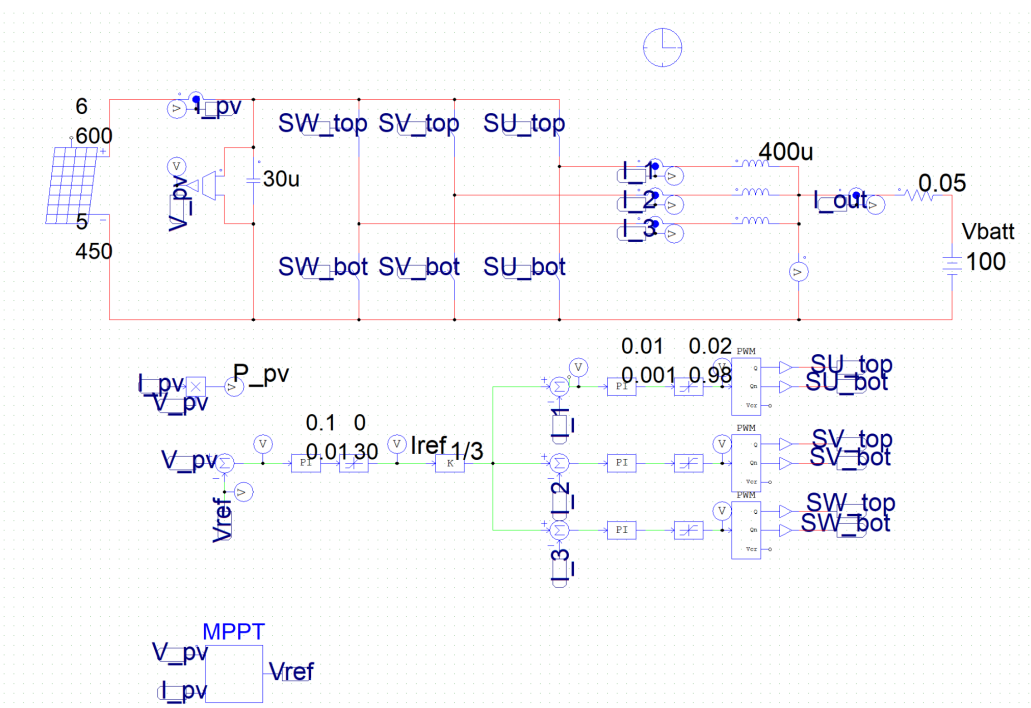


Figure 11: Perturb and Observe code implemented into C-Block. (See Appendix A)

```c
#include <Stdlib.h>
#include <String.h>
#include <math.h>
#include <Psim.h>

// 1. PLACE GLOBAL/STATIC VARIABLES HERE
static float Vprev = 0.0;
static float Iprev = 0.0;
static float Pprev = 0.0;
static float Vref = 599.0;
const float step = 0.01;

void SimulationStep(
    double t, double deltaT, double *in, double *out,
    int *pnError, char * szErrorMsg,
    void ** reserved_UserData, int reserved_ThreadIndex, void * reserved_AppPtr)
{
    // 2. MAP INPUTS (x1 = in[0], x2 = in[1])
    float V_pv = (float)in[0];
    float I_pv = (float)in[1];
    float Ppv = V_pv * I_pv;

    float Pdiff = Ppv - Pprev;
    float Vdiff = V_pv - Vprev;

    // 3. YOUR MPPT LOGIC
    if (fabs(Pdiff) > 0.01) {
      if (Pdiff > 0) {
        if (Vdiff > 0) {
          Vref += step;
        } else {
          Vref -= step;
        }
      } else if (Pdiff < 0) {
        if (Vdiff > 0) {
          Vref -= step;
        } else {
          Vref += step;
        }
      }
    }

    // 4. UPDATE STATES & MAP OUTPUT (y1 = out[0])
    Vprev = V_pv;
    Iprev = I_pv;
    Pprev = Ppv;

    out[0] = (double)Vref;
}

void SimulationBegin(
    const char *szId, int nInputCount, int nOutputCount,
    int nParameterCount, const char ** pszParameters,
    int *pnError, char * szErrorMsg,
    void ** reserved_UserData, int reserved_ThreadIndex, void * reserved_AppPtr)
{
    // Reset variables at the start of every simulation run
    Vprev = 0.0;
    Iprev = 0.0;
    Pprev = 0.0;
    Vref = 599.0;
}

void SimulationEnd(const char *szId, void ** reserved_UserData, int reserved_ThreadIndex, void * reserved_AppPtr)
{
    // Leave empty unless you are doing advanced memory management
}
```

Figure 12: Ppv, Vpv, and Ipv plot vs time for MPPT schematic.

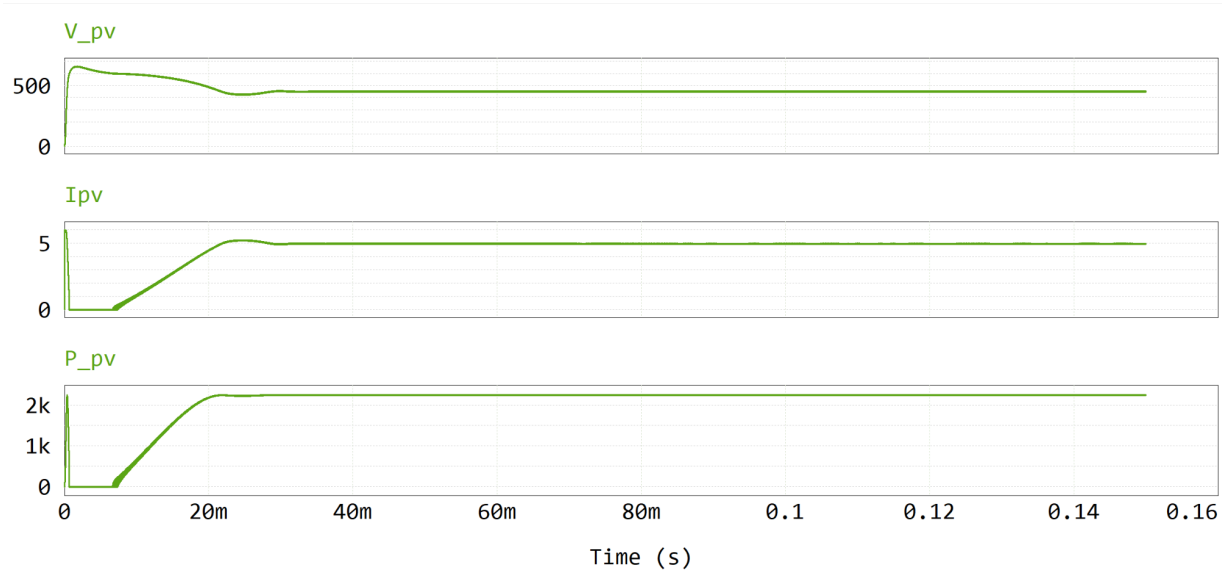


Figure 11 validates the MPPT tracking algorithm as the system reaches a steady state output at about 2.25KW which matches out expected max power output from the PV characterization from **Figure 3.** Additionally we can see that Vpv and Ipv track accordingly their specific MP rating of 450Vdc and 5A respectively.

Conclusion

This PSIM simulation of an interleaved buck-boost converter and MPPT algorithm successfully demonstrated efficient power conversion generated from PV panels to a 100V battery. The final schematic consists of effective current control, voltage control, and an MPPT algorithm to provide stable and efficient energy delivery from our PV system. The perturbation and observation algorithm was implemented for the MPPT by adjusting the converter's duty cycle to track variation in our simulated solar irradiance. The results from Figures 5, 8 and 12 validate our systems ability to track to desired operated setpoints and maintain operations at maximum power. Overall the project demonstrates the importance of closed-loop control strategies in solar applications that can improve the efficiency and reliability of renewable energy sources.

Each team member contributed significantly to this project's success including circuit design, data collection, background research and report writing. Charlie was the lead in preliminary circuit design and controller setting ensuring the current and voltage schematics were designed appropriately and ready to be tuned. Charlie also played a large role in the writing of the MPPT algorithm and implementation into the MPPT PSIMM schematic. Nolan and Barry contributed to the current, voltage, and MPPT algorithm by adjusting controller values to achieve more favorable overshoot values and settling times. Barry led the report writing for the introduction and problem definition. Nolan led the report writing in documenting the results from the PSIM schematics and creating appropriate figures to include in the solutions sections.

Appendix A: C Block Code for MPPT control

```
#include <Stdlib.h>
#include <String.h>
#include <math.h>
#include <Psim.h>

// GLOBAL vars
static float Vprev = 0.0;
static float Iprev = 0.0;
static float Pprev = 0.0;
static float Vref = 599.0;
const float step = 0.01;

void SimulationStep(
   double t, double delt, double *in, double *out,
   int *pnError, char * szErrorMsg,
   void ** reserved_UserData, int reserved_ThreadIndex, void * reserved_AppPtr)
{
   // 2. Mapping INPUTS
   float V_pv = (float)in[0];
   float I_pv = (float)in[1];
   float Ppv = V_pv * I_pv;

   float Pdiff = Ppv - Pprev;
   float Vdiff = V_pv - Vprev;

   // MPPT control
   if (fabs(Pdiff) > 0.01) {
      if (Pdiff > 0) {
         if (Vdiff > 0) {
            Vref += step;
         } else {
            Vref -= step;
         }
      } else if (Pdiff < 0) {
         if (Vdiff > 0) {
            Vref -= step;
         } else {
            Vref += step;
         }
      }
   }

   // Update vars and map OUTPUT
   Vprev = V_pv;
   Iprev = I_pv;
   Pprev = Ppv;

   out[0] = (double)Vref;
}
// PSIM included code in C block
void SimulationBegin(
   const char *szId, int nInputCount, int nOutputCount,
   int nParameterCount, const char ** pszParameters,
   int *pnError, char * szErrorMsg,
   void ** reserved_UserData, int reserved_ThreadIndex, void * reserved_AppPtr)
{
   // Reset variables at the start of every simulation run
   Vprev = 0.0;
   Iprev = 0.0;
   Pprev = 0.0;
   Vref = 599.0;
}

void SimulationEnd(const char *szId, void ** reserved_UserData, int reserved_ThreadIndex, void * reserved_AppPtr)
{
   // Leave empty unless you are doing advanced memory management
}
```